```
!pip install -U tensorflow-addons
```

```
Requirement already satisfied: tensorflow-addons in /usr/local/lib/python3.7/dist-packages (0.14.0)
Requirement already satisfied: typeguard>=2.7 in /usr/local/lib/python3.7/dist-packages (from tensorflow-addons) (2.7.1)
```

```
#Importing Required Libraries

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn.metrics import f1_score,accuracy_score
```

```
#References 1 : https://stackoverflow.com/questions/46000191/utf-8-codec-cant-decode-byte-0x92-in-position-18-invalid-start-byte
#References 2 : https://docs.python.org/2.4/lib/standard-encodings.html
```

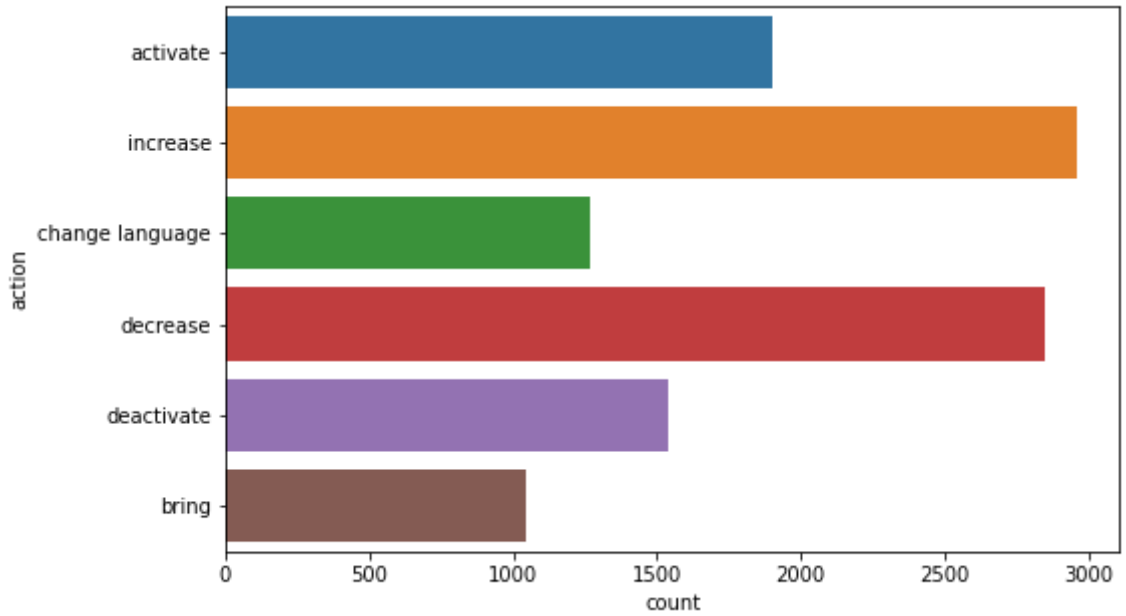
```
trainData = pd.read_csv('/content/train_data.csv',encoding='cp1252')
validData = pd.read_csv('/content/valid_data.csv',encoding='cp1252')
trainData.head()
```

	path	transcription	action	object	location
0	wavs/speakers/xRQE5VD7rRHVdyvM/7372ca00-45c4-1...	Turn on the kitchen lights	activate	lights	kitchen
1	wavs/speakers/R3mexpM2YAtdPbL7/dae28110-44fe-1...	Turn up the temperature	increase	heat	none
2	wavs/speakers/ZebMRl5Z7dhrPKRD/b55dcfd0-455d-1...	OK now switch the main language to Chinese	change language	Chinese	none
3	wavs/speakers/ppzZqYxGkESMdA5Az/61c54a20-4476-...	Turn down the bathroom temperature	decrease	heat	washroom
4	wavs/speakers/zaEBPeMY4NUbDnZy/8ef57ec0-44df-1...	Change the language	change language	none	none

```
trainData.columns
```

```
Index(['path', 'transcription', 'action', 'object', 'location'], dtype='object')
```

```
#count of the labels/action
fig, ax = plt.subplots(figsize=(8,5))
sns.countplot(data=trainData,y='action')
plt.show()
```




```
#Splitting the data

y_trainData = trainData['action']
X_trainData = trainData.drop(['action'],axis=1)
y_validData = validData['action']
X_validData = validData.drop(['action'],axis=1)

print(X_trainData.shape)
print(X_validData.shape)
print(y_trainData.shape)
print(y_validData.shape)
```

```
(11566, 4)
(3118, 4)
(11566,)
(3118,)
```

```
#Analyzing text data for padding
#Getting the median of length of the input/transcription

lengthOfText = []
for i in range(X_trainData.shape[0]):
  lengthOfText.append(len(X_trainData['transcription'].iloc[i]))
np.median(lengthOfText)
```

```
20.0
```

```
#Tokenizing the sentences/Transcription
tokenizer = Tokenizer(num_words=100,filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n',split=' ')
tokenizer.fit_on_texts(list(X_trainData['transcription']))

#Converting text into integer sequences
seq_train = tokenizer.texts_to_sequences(X_trainData['transcription'])
seq_val = tokenizer.texts_to_sequences(X_validData['transcription'])

#padding to prepare sequences
seq_train = pad_sequences(sequences=seq_train,maxlen=1000)
seq_val = pad_sequences(sequences=seq_val,maxlen=1000)
```

```
vocab_size = len(tokenizer.word_index) + 1  #here +1 is for padding
print(vocab_size)
print(tokenizer.word_index)
```

```
99
{'the': 1, 'turn': 2, 'in': 3, 'lights': 4, 'on': 5, 'up': 6, 'down': 7, 'heat': 8, 'temperature': 9, 'switch': 10, 'kitchen':
```

```
#Loading glove vectors for word embedding

import pickle
with open('/content/glove_vectors','rb') as f:
  glove_words = pickle.load(f)


matrix1 = np.zeros((vocab_size,300))
for word,rank in tokenizer.word_index.items():
  vec = glove_words.get(word)
  if vec is not None:
    matrix1[rank] = vec
print(matrix1)
```

```
[[ 0.         0.         0.        ...  0.         0.         0.       ]
 [-0.20838   -0.14932   -0.017528  ... -0.54066    0.21199   -0.0094357]
 [-0.49485    0.017693  -0.18999   ...  0.13083    0.11391   -0.076012 ]
 ...
 [-0.46529    0.1063    -0.29215   ...  0.19393   -0.25652    0.26225  ]
 [ 0.11396    0.34503   -0.055064  ... -0.37247    0.2879    -0.18029  ]
 [ 0.27502   -0.52118   -0.1479    ...  0.22491    0.27388   -0.13467  ]]
```

```
#preprocessing using OneHotEncoder
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder()
encoded_y_trainData = ohe.fit_transform(np.array(y_trainData).reshape(-1,1))
encoded_y_validData = ohe.fit_transform(np.array(y_validData).reshape(-1,1))

encoded_y_trainData = encoded_y_trainData.toarray()
encoded_y_validData = encoded_y_validData.toarray()

print(encoded_y_trainData.shape)
print(encoded_y_validData.shape)
```

```
(11566, 6)
(3118, 6)
```

```
#importing required tensorflow libraries

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.callbacks import Callback
from tensorflow.keras.callbacks import LearningRateScheduler
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.layers import Dense,Input,Activation,BatchNormalization,Dropout,Embedding,Flatten,Conv1D,MaxPool1D,concatenate
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import Callback
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import tensorflow_addons as tfa


#CNN model

#creating an input layer
input_1 = tf.keras.Input(shape=(1000,))

#Now we are creating an embedding layer
embedd = Embedding(vocab_size,300,weights =[matrix1],input_length=1000,trainable=False)(input_1)

#1st conv layer
Conv1_m = Conv1D(filters=20,kernel_size=2,strides=1,activation='relu',kernel_initializer=tf.keras.initializers.he_normal,kernel_regul
#Conv Layer 2
Conv1_n= Conv1D(filters=16,kernel_size=2,strides=1,activation='relu',kernel_initializer=tf.keras.initializers.he_normal,kernel_regula
#conv Layer 3
Conv1_o = Conv1D(filters=12,kernel_size=2,strides=1,activation='relu',kernel_initializer=tf.keras.initializers.he_normal,kernel_regul

#Concatenating all 3 1D conv layers
concat_1 = concatenate([Conv1_m,Conv1_n,Conv1_o])
#adding a dropout of rate 0.2
dropout_1 = Dropout(rate=0.2)(concat_1)
#adding a batch normalisation layer to the dropout layer
```

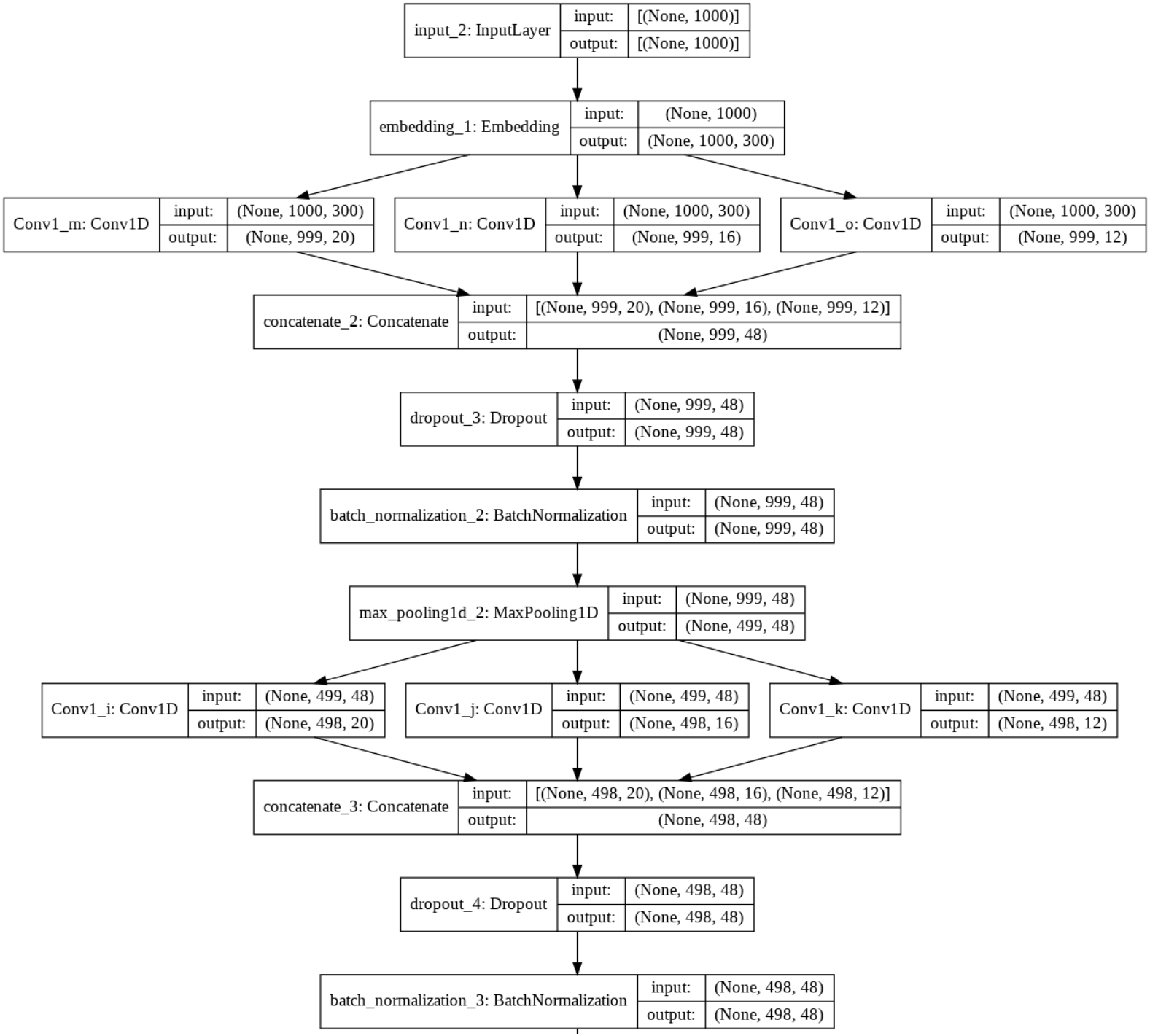
```
#we must add batch norm + dropout so as to avoid internal covariance shift
batchNorm_1 = BatchNormalization(momentum=0.9)(dropout_1)

#adding maxpool layer
maxpool_1 = MaxPool1D(pool_size=2,strides=2)(batchNorm_1)

#Conv Layer 1
Conv1_i = Conv1D(filters=20,kernel_size=2,strides=1,activation='relu',kernel_initializer=tf.keras.initializers.he_normal,kernel_regul
#Conv Layer 2
Conv1_j= Conv1D(filters=16,kernel_size=2,strides=1,activation='relu',kernel_initializer=tf.keras.initializers.he_normal,kernel_regula
#conv Layer 3
Conv1_k = Conv1D(filters=12,kernel_size=2,strides=1,activation='relu',kernel_initializer=tf.keras.initializers.he_normal(seed=36),ker

#Concatenating all 3 1D conv layers
concat_2 = concatenate([Conv1_i,Conv1_j,Conv1_k])
#adding a dropout of rate 0.2
dropout_2 = Dropout(rate=0.2)(concat_2)
#adding a batch normalisation layer to the dropout layer
batchNorm_2 = BatchNormalization(momentum=0.9)(dropout_2)
#adding maxpool layer
maxpool_2 = MaxPool1D(pool_size=2,strides=2)(batchNorm_2)
#Conv Layer 1
Conv1_p = Conv1D(filters=20,kernel_size=2,strides=1,activation='relu',kernel_initializer=tf.keras.initializers.he_normal,kernel_regul

#flattening
flatten_1 = Flatten(data_format='channels_last')(Conv1_p)
dropout_3 = Dropout(rate=0.2)(flatten_1)
#dense layer
dense_1 = Dense(32,activation='relu',kernel_initializer=tf.keras.initializers.he_normal,kernel_regularizer=l2(0.01))(dropout_3)
#output layer
output_1 = Dense(units=6,activation='softmax',kernel_initializer=tf.keras.initializers.glorot_normal)(dense_1)



#Model summary
```

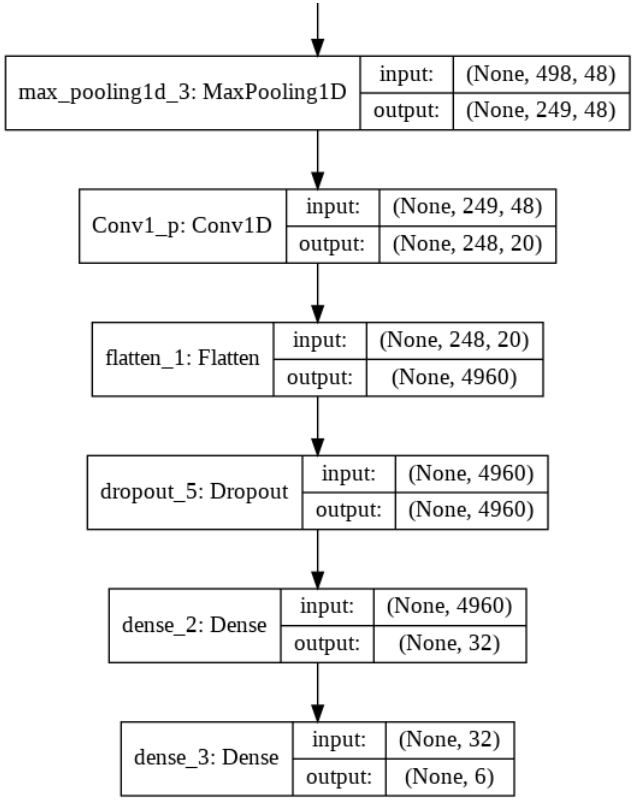
```
my_model = Model(inputs=input_1, outputs=output_1)
my_model.summary()
```

```
Model: "model_1"
__________________________________________________________________________________________________
Layer (type)                    Output Shape         Param #     Connected to
==================================================================================================
input_2 (InputLayer)            [(None, 1000)]       0
__________________________________________________________________________________________________
embedding_1 (Embedding)         (None, 1000, 300)    29700       input_2[0][0]
__________________________________________________________________________________________________
Conv1_m (Conv1D)                (None, 999, 20)      12020       embedding_1[0][0]
__________________________________________________________________________________________________
Conv1_n (Conv1D)                (None, 999, 16)      9616        embedding_1[0][0]
__________________________________________________________________________________________________
Conv1_o (Conv1D)                (None, 999, 12)      7212        embedding_1[0][0]
__________________________________________________________________________________________________
concatenate_2 (Concatenate)     (None, 999, 48)      0           Conv1_m[0][0]
                                                                 Conv1_n[0][0]
                                                                 Conv1_o[0][0]
__________________________________________________________________________________________________
dropout_3 (Dropout)             (None, 999, 48)      0           concatenate_2[0][0]
__________________________________________________________________________________________________
batch_normalization_2 (BatchNor (None, 999, 48)      192         dropout_3[0][0]
__________________________________________________________________________________________________
max_pooling1d_2 (MaxPooling1D)  (None, 499, 48)      0           batch_normalization_2[0][0]
__________________________________________________________________________________________________
Conv1_i (Conv1D)                (None, 498, 20)      1940        max_pooling1d_2[0][0]
__________________________________________________________________________________________________
Conv1_j (Conv1D)                (None, 498, 16)      1552        max_pooling1d_2[0][0]
__________________________________________________________________________________________________
Conv1_k (Conv1D)                (None, 498, 12)      1164        max_pooling1d_2[0][0]
__________________________________________________________________________________________________
concatenate_3 (Concatenate)     (None, 498, 48)      0           Conv1_i[0][0]
                                                                 Conv1_j[0][0]
                                                                 Conv1_k[0][0]
__________________________________________________________________________________________________
dropout_4 (Dropout)             (None, 498, 48)      0           concatenate_3[0][0]
__________________________________________________________________________________________________
batch_normalization_3 (BatchNor (None, 498, 48)      192         dropout_4[0][0]
__________________________________________________________________________________________________
```

```
max_pooling1d_3 (MaxPooling1D)  (None, 249, 48)      0           batch_normalization_3[0][0]
__________________________________________________________________________________________________
Conv1_p (Conv1D)                (None, 248, 20)      1940        max_pooling1d_3[0][0]
__________________________________________________________________________________________________
flatten_1 (Flatten)             (None, 4960)         0           Conv1_p[0][0]
__________________________________________________________________________________________________
dropout_5 (Dropout)             (None, 4960)         0           flatten_1[0][0]
__________________________________________________________________________________________________
dense_2 (Dense)                 (None, 32)           158752      dropout_5[0][0]
__________________________________________________________________________________________________
dense_3 (Dense)                 (None, 6)            198         dense_2[0][0]
==================================================================================================
Total params: 224,478
Trainable params: 194,586
Non-trainable params: 29,892
```

```
from tensorflow.keras.utils import plot_model
plot_model(my_model, 'model1WithShapes.png', show_shapes=True)
```


input_2: InputLayer
input: [(None, 1000)]
output: [(None, 1000)]
embedding_1: Embedding
input: (None, 1000)
output: (None, 1000, 300)
Conv1_m: Conv1D
input: (None, 1000, 300)
output: (None, 999, 20)
Conv1_n: Conv1D
input: (None, 1000, 300)
output: (None, 999, 16)
Conv1_o: Conv1D
input: (None, 1000, 300)
output: (None, 999, 12)
concatenate_2: Concatenate
input: [(None, 999, 20), (None, 999, 16), (None, 999, 12)]
output: (None, 999, 48)
dropout_3: Dropout
input: (None, 999, 48)
output: (None, 999, 48)
batch_normalization_2: BatchNormalization
input: (None, 999, 48)
output: (None, 999, 48)
max_pooling1d_2: MaxPooling1D
input: (None, 999, 48)
output: (None, 499, 48)
Conv1_i: Conv1D
input: (None, 499, 48)
output: (None, 498, 20)
Conv1_j: Conv1D
input: (None, 499, 48)
output: (None, 498, 16)
Conv1_k: Conv1D
input: (None, 499, 48)
output: (None, 498, 12)
concatenate_3: Concatenate
input: [(None, 498, 20), (None, 498, 16), (None, 498, 12)]
output: (None, 498, 48)
dropout_4: Dropout
input: (None, 498, 48)
output: (None, 498, 48)
batch_normalization_3: BatchNormalization
input: (None, 498, 48)
output: (None, 498, 48)


max_pooling1d_3: MaxPooling1D
input:
(None, 498, 48)
output:
(None, 249, 48)
Conv1_p: Conv1D
input:
(None, 249, 48)
output:
(None, 248, 20)
flatten_1: Flatten
input:
(None, 248, 20)
output:
(None, 4960)
dropout_5: Dropout
input:
(None, 4960)
output:
(None, 4960)
dense_2: Dense
input:
(None, 4960)
output:
(None, 32)
dense_3: Dense
input:
(None, 32)
output:
(None, 6)

```
%load_ext tensorboard
```

```
The tensorboard extension is already loaded. To reload it, use:
  %reload_ext tensorboard
```

```
class TerminateNaN(tf.keras.callbacks.Callback):

    def on_epoch_end(self, epoch, logs={}):
        loss = logs.get('loss')
        model=logs.get('model')
        weigh=np.array(self.model.get_weights())
        if loss is not None:
            if np.isnan(loss) or np.isinf(loss):
                print("Invalid loss and terminated at epoch {}".format(epoch))
                self.model.stop_training = True
        if weigh is not None:
           if pd.isnull(weigh).any():
              print("Invalid loss and terminated at epoch {}".format(epoch))
              self.model.stop_training = True
```

```
#removing logs
!rm -rf ./logs/
```

```
import datetime

tf.debugging.set_log_device_placement(True)
fpath = 'best_model_L1.h5'
checkpoint = ModelCheckpoint(filepath=fpath, monitor='val_accuracy',  verbose=1, save_best_only=True, mode='auto')
earlystop = EarlyStopping(monitor='val_accuracy',min_delta=0.0001,patience=2,verbose=1)
quitNAN = TerminateNaN()
#history_own = LossHistory()
callbacklist = [checkpoint,earlystop,quitNAN]
optimizer=tf.keras.optimizers.Adam(learning_rate=0.001)
```

```
log_dir = 'logs/fit/'+datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=log_dir,histogram_freq=1, write_graph=True,write_grads=True)
```

```
my_model.compile(optimizer=optimizer, loss='categorical_crossentropy',metrics=['accuracy',tfa.metrics.F1Score(num_classes=20,average=
my_model.fit(seq_train,encoded_y_trainData,epochs=50,validation_data=(seq_val,encoded_y_validData),callbacks=[callbacklist,tensorboar
```

```
WARNING:tensorflow:`write_grads` will be ignored in TensorFlow 2.0 for the `TensorBoard` Callback.
Epoch 1/50
362/362 [==============================] - 131s 355ms/step - loss: 2.1258 - accuracy: 0.6910 - f1_score: 0.6910 - val_loss: 1.0

Epoch 00001: val_accuracy improved from -inf to 0.84606, saving model to best_model_L1.h5
/usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:6: VisibleDeprecationWarning: Creating an ndarray from ragged nest

Epoch 2/50
362/362 [==============================] - 128s 354ms/step - loss: 0.8533 - accuracy: 0.8250 - f1_score: 0.8250 - val_loss: 0.6

Epoch 00002: val_accuracy did not improve from 0.84606
Epoch 3/50
362/362 [==============================] - 126s 349ms/step - loss: 0.5426 - accuracy: 0.8376 - f1_score: 0.8376 - val_loss: 0.4

Epoch 00003: val_accuracy improved from 0.84606 to 0.84702, saving model to best_model_L1.h5
Epoch 4/50
362/362 [==============================] - 129s 356ms/step - loss: 0.4481 - accuracy: 0.8322 - f1_score: 0.8322 - val_loss: 0.3

Epoch 00004: val_accuracy did not improve from 0.84702
Epoch 5/50
362/362 [==============================] - 128s 354ms/step - loss: 0.4054 - accuracy: 0.8374 - f1_score: 0.8374 - val_loss: 0.3

Epoch 00005: val_accuracy did not improve from 0.84702
Epoch 00005: early stopping
<keras.callbacks.History at 0x7fd339cae750>
```

```
%tensorboard --logdir logs/fit
```

TensorBoard SCALARS GRAPHS DISTRIBUTIONS HISTOGRAMS TIME SE INACTIVE

Show data download links

Ignore outliers in chart scalin

Tooltip sorting method: default

Smoothing

0.6

Horizontal Axis

STEP RELATIVE

WALL

Runs

Write a regex to filter runs

20211027-020326/train

20211027-020326/validation

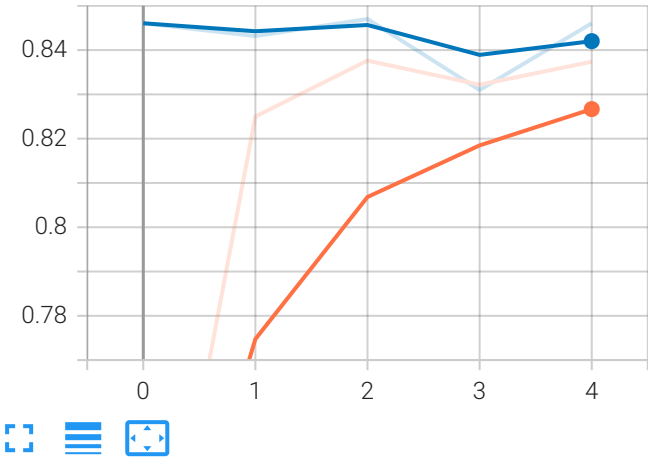
TOGGLE ALL RUNS

logs/fit

Filter tags (regular expressions supported)

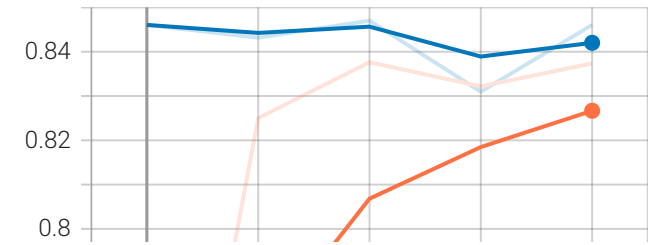
epoch_accuracy

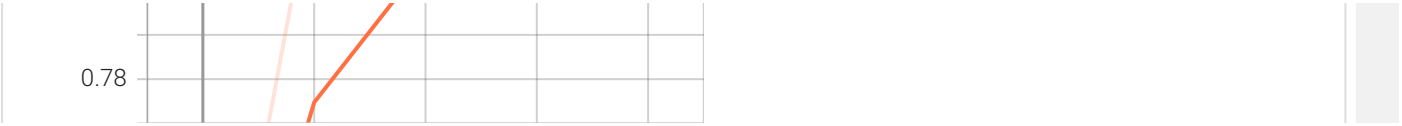
epoch_accuracy
tag: epoch_accuracy



epoch_f1_score

epoch_f1_score
tag: epoch_f1_score




0.78